

UCS503 — Software Engineering

LifeAdmin

*A Personal Administrative Management Platform
with Document-Based Renewal and Due-Date Detection*

Submitted by:

1024030336 Kushant Garg

1024031191 Sanvi

1024030323 Loveleen Singh

Group: 3C23, Computer Engineering (COE)

Submitted to:

Dr. Raghav B. Venkataramaiyer

Progress Report

Computer Science and Engineering Department
Thapar Institute of Engineering and Technology, Patiala

Contents

Chapter 1: Introduction	3
1.1 Background and Context	3
1.1.1 Problem Statement	3
1.2 Target Users and Use Cases	3
1.3 Project Objectives	4
1.4 Scope and Limitations	4
Chapter 2: Methodology and Design	4
2.1 System Architecture	4
2.1.1 High-Level Design	5
2.1.2 Technical Stack	5
2.1.3 Data Model	5
2.2 Implementation Details	6
2.2.1 Module 1: Core Functionality	6
2.2.2 Module 2: Secondary and Planned Features	6
2.3 Development Process	7
Chapter 3: Results and Evaluation	7
3.1 Testing Methodology	7
3.2 Manual Verification Checklist	7
3.3 Performance Metrics	8
3.4 Analysis	8
3.5 Risk Assessment	9
Chapter 4: Conclusion	9
4.1 Summary of Work	9
4.2 Key Findings	10
4.3 Future Work and Recommendations	10
4.4 Final Remarks	10
4.5 Acknowledgements	11
Appendix A: Source File Summary	11
Bibliography	12

Chapter 1: Introduction

1.1 Background and Context

Individuals routinely lose track of personal administrative deadlines and renewals — ID and licence renewals, insurance policies, subscription cycles, loan EMI schedules, and similar recurring obligations. Document-locker services such as DigiLocker solve the problem of storing a document, but they do not solve the problem of acting on the obligation tied to that document — a stored insurance policy still requires the owner to separately remember when it expires. LifeAdmin is a JavaFX desktop application that targets this "document-action gap": it combines a personal deadline tracker with a planned document-scanning feature that reads a date directly off an uploaded document (an insurance renewal notice, a licence, an EMI schedule) and turns it into a tracked, reminder-backed obligation, rather than leaving that step to the user's memory.

The underlying idea is that a document and the obligation it represents are really two views of the same event: an insurance policy PDF is also, implicitly, a renewal-date reminder; a licence scan is also an expiry-date reminder. Most tools today treat storage and reminders as separate products — a locker app for the former, a calendar or notes app for the latter — and require the user to manually bridge the two every time a new document arrives. LifeAdmin's long-term goal is to let the document itself populate the reminder.

1.1.1 Problem Statement

Students and individuals face several recurring problems in tracking administrative obligations:

- **Fragmentation:** obligations are scattered across emails, paper notices, apps, and memory, with no single place to see what is due.
- **Document-action gap:** locker-style platforms give access to a document but do not track or remind about the action required around it (e.g., an expiry date).
- **Reactive, not proactive, awareness:** a missed deadline is typically discovered only after a penalty, late fee, or service disruption.
- **Poor task-metadata linkage:** even when reminders exist (calendar entries, notes apps), they are not linked to the relevant service or renewal cadence, so they are easy to dismiss or mis-set.

LifeAdmin is designed to close this gap for individual users through a dedicated administrative dashboard and, as the project's centerpiece feature, automatic date extraction from uploaded documents.

1.2 Target Users and Use Cases

The application targets individuals who juggle a small but recurring set of personal administrative obligations rather than an organization managing many people's records. Representative use cases include:

- **A university student** tracking hostel/mess dues, a driving licence renewal, and a two-wheeler insurance policy, each with a different renewal cadence.

- **A salaried individual** tracking multiple loan EMI schedules, a health-insurance renewal, and subscription-style payments (rent, utilities) alongside one-off obligations.
- **Any household member** who currently relies on scattered paper notices or bank SMS alerts for renewal dates and wants a single dashboard that also stores the underlying document.

In each case, the common thread is a small number of recurring, date-bound obligations that are individually easy to track but collectively easy to lose in email or paper — exactly the gap described in section 1.1.1.

1.3 Project Objectives

1. **Objective 1 (delivered):** build a functional JavaFX + JDBC desktop application supporting user registration/login, obligation ("chore") entry with due date, category and recurrence, and a dashboard that lists obligations and flags those due within 7 days.
2. **Objective 2 (in progress):** achieve an on-time completion rate of at least 85% among pilot users' tracked obligations, measured from due-date versus completion-timestamp data already captured in the database.
3. **Objective 3 (core remaining scope):** implement the document-upload and OCR date-detection pipeline — cropping a known region of a supported document template (e.g., an insurance policy, a licence, an EMI schedule) and running OCR to suggest a renewal date, always subject to user confirmation before a chore is created — and validate it with a pilot of 10–20 users.

These objectives follow SMART criteria: they are specific to the LifeAdmin obligation-tracking workflow, measurable via on-time completion rate and OCR-acceptance rate, achievable within the semester given the team's developing Java/JavaFX/JDBC skillset, relevant to the document-action gap described above, and time-bound to the current milestone and pilot window.

1.4 Scope and Limitations

To keep the project achievable within a semester, the current scope is deliberately narrow in a few ways. It targets a single desktop user session at a time rather than a multi-device or mobile experience; it stores documents on the local filesystem rather than in the cloud; and the planned OCR pipeline is scoped to a small, curated set of document layouts (starting with one or two templates) rather than general-purpose document understanding. Government integrations such as a live DigiLocker API connection, multi-user household accounts, and push notifications to a phone are explicitly out of scope for this milestone and are noted as possible extensions rather than committed deliverables.

Chapter 2: Methodology and Design

2.1 System Architecture

LifeAdmin is a layered desktop application rather than a client–server web app: a JavaFX front end, a plain-Java domain layer, and a JDBC persistence layer. This keeps the UI, the business logic (obligation and recurrence handling, and eventually OCR-to-obligation confirmation), and the storage layer independently editable, which matters as the document-scanning feature is added on top of the existing chore-tracking core.

2.1.1 High-Level Design

The application is organized around four parts: (a) an authentication module (UserDAO) for registration and login, so each user's data is isolated; (b) an obligation module (Chore, ChoreDAO) that stores a title, due date, category, completion flag, and recurrence rule, and automatically schedules the next cycle for a recurring item once it is completed; (c) a document module (Document, DocumentDAO) that currently stores and lists uploaded files against a user; and (d) a JavaFX dashboard that ties these together in a tabbed "Chores" / "Documents" view and visually flags any chore due within the next 7 days.

2.1.2 Technical Stack

- **Framework/Language:** JavaFX for the desktop UI; plain Java for domain and business logic.
- **Database:** the initial proposal specified JDBC + SQLite for a self-contained, zero-setup deployment; the implementation pivoted to JDBC + MySQL (via the `mysql-connector-j` driver) for the current schema in `Database.java`, `ChoreDAO.java`, `DocumentDAO.java`, and `UserDAO.java`.
- **Planned OCR:** Tess4J, a Java wrapper for the open-source Tesseract engine, scoped to run only on a cropped, pre-defined region of a small set of supported document templates rather than full-document OCR — not yet integrated into the codebase.
- **Tools:** Git/GitHub for version control. A CI pipeline (GitHub Actions with JUnit) is planned but has not yet been set up in the repository.

2.1.3 Data Model

The schema currently consists of three MySQL tables, created on startup by `Database.createTable()`, `UserDAO.createUserTable()`, and `DocumentDAO.createTable()` respectively:

Table	Key Columns	Notes
users	id (PK, auto-increment), username (unique), password	Password is stored and compared as plain text (see 3.4).
chores	id (PK), title, due_date, category, is_done, document_path, user_id (FK-like), recurrence	due_date is a VARCHAR (ISO string), not a native DATE column. document_path is a free-text field, not a real foreign key to documents.id.
documents	id (PK), title, file_path, upload_date, user_id (FK-like)	Populated by the Documents tab; not currently referenced from chores.document_path in the UI.

Table 2.1: Current database schema (three tables, no formal foreign-key constraints)

Two design points stand out for the OCR work planned next. First, `due_date` and `upload_date` are stored as VARCHAR rather than a native DATE/DATETIME type; this works today because the application always writes and reads ISO-formatted strings (via `LocalDate.toString()` / `LocalDate.parse()`), but it forfeits database-level date validation and native date arithmetic. Second, chores already has a `document_path` column intended to link an obligation to its source document, but the current UI always inserts a chore with this field set to null, and the standalone documents table is never joined against it — so,

structurally, the schema anticipates the OCR-to-obligation link the project still needs to build, but the two modules are not yet wired together.

2.2 Implementation Details

2.2.1 Module 1: Core Functionality

The core, currently implemented workflow covers a login/registration screen, a tabbed dashboard, and full obligation management:

- A login screen (`ChoreApp.showLoginScreen`) with username/password fields and Login/Register buttons; registration calls `UserDAO.register` and reports "Username already taken" whenever the insert fails, relying on the `users.username` UNIQUE constraint rather than a separate existence check.
- A tabbed dashboard (`ChoreApp.showDashboard`) with "Chores" and "Documents" navigation buttons and a welcome label showing the logged-in username.
- An obligation model and DAO (`ChoreDAO`) with `addChore`, `getAllChores`, `getChoresDueSoon`, `markDone` and `deleteChore`, plus a `completeChore` method that marks an item done and, if it carries a weekly, monthly, or yearly recurrence, automatically inserts the next cycle's row with the recalculated due date.
- An add-obligation form (title, due-date picker, recurrence dropdown: none/weekly/monthly/yearly) that always saves the category as the fixed string "General" — the `Chore` model and `chores.category` column support categorization, but the UI does not yet expose a category selector to the user.
- A live list view that visually flags any obligation due within 7 days with an amber accent border, and offers per-item "Mark Done" and "Delete" actions — the "administrative dashboard" and "deadline tracking" pieces of the original proposal.

2.2.2 Module 2: Secondary and Planned Features

Implemented: a Documents tab lets a user pick a file (JavaFX `FileChooser`), give it a title, and upload it. The file is copied into a per-user-machine storage folder (`~/LifeAdminDocuments`) under a timestamp-prefixed name to avoid collisions, and `DocumentDAO.addDocument` records the title, stored path, and upload date. Users can list, open (via the OS's default application through `java.awt.Desktop`), and delete their uploaded documents.

Planned — and the centerpiece of the project's remaining scope: automatic date detection. The intended pipeline has five stages: (1) match the uploaded file against a small library of known document templates; (2) crop the pre-defined region of that template where the relevant date is expected to appear; (3) run Tess4J OCR on just that cropped region rather than the full page, to keep accuracy high and processing fast; (4) parse the OCR text into a candidate date and attach a confidence score; and (5) always show the detected date back to the user in a confirmation dialog before a chore is created — the user can accept, edit, or discard the suggestion. Unsupported document types, or failed/low-confidence reads, will fall back to the existing manual-entry form. Building this pipeline will also require closing the data-model gap noted in section 2.1.3, so that a chore created this way is properly linked back to the document it came from. A background reminder scheduler (proposed as a `ScheduledExecutorService` job

with 30/7/1-day lead times) is also planned but not yet built; the current build surfaces "due soon" only as a UI flag on the dashboard, not as a push-style notification.

2.3 Development Process

The team of three (Kushant Garg, Sanvi, Loveleen Singh) is working under the guidance of Dr. Raghav B. Venkataramaiyer, following a weekly progress-journal cadence — the repository's docs/journal folder records each week's goals, work completed, and blockers. The first week's journal frames the milestone as getting a runnable skeleton in place: a working login/registration flow, basic obligation CRUD, and a first pass at the JavaFX dashboard, all backed by a real database rather than in-memory placeholders. Version control is via Git/GitHub with the source organized under a single `com.lifeadmin` package rather than split into UI/domain/persistence sub-packages, which is a reasonable simplification at the current codebase size but is worth revisiting as the OCR module adds more classes.

Chapter 3: Results and Evaluation

3.1 Testing Methodology

Testing to date has been manual rather than automated. Each ChoreDAO method (add, get, complete, delete, getChoresDueSoon) was exercised against a live MySQL database during development, and the login/registration and document upload/list/delete flows were checked end to end through the JavaFX UI. The proposal's plan for automated JUnit tests on scheduling and OCR-region logic, run through a CI pipeline on every commit, has not yet been set up; this gap is called out explicitly in section 4.3 below.

3.2 Manual Verification Checklist

The table below lists the core flows exercised during development and their status, as reconstructed from the current codebase and the team's week-1 journal:

Flow	Exercised During Development
Register a new account / reject duplicate username	Yes — UNIQUE constraint on <code>users.username</code> surfaces as a "username already taken" message
Login with correct / incorrect credentials	Yes
Add a chore with no recurrence	Yes
Add a chore with weekly / monthly / yearly recurrence and mark it done	Yes — <code>completeChore()</code> inserts the next cycle automatically
Delete a chore	Yes
"Due soon" (7-day) highlighting on the dashboard	Yes
Upload a document and confirm it is copied into local storage	Yes

Open an uploaded document from the list	Yes
Delete an uploaded document	Yes
Automatic OCR date detection from an uploaded document	Not applicable — feature not yet built

Table 3.2: Manually exercised flows as of this milestone

3.3 Performance Metrics

The table below summarizes the metrics defined for evaluating LifeAdmin and their status at this stage of development:

Metric	Target	Achieved
On-time completion rate	$\geq 85\%$ of tracked obligations completed before their due date	Due-date and completion timestamps are captured in the database; aggregation/reporting not yet built
Reminder delivery reliability	Scheduled reminders surfaced before the due date	Partial — 7-day "due soon" flag implemented in the dashboard UI; background scheduler not yet built
OCR suggestion accuracy	% of OCR-suggested dates accepted without correction	Not yet measurable — document-scanning/OCR pipeline not yet implemented
Core workflow reliability	App completes add/view/complete/delete flows without crashing	Achieved for chores and documents under manual testing

Table 3.3: LifeAdmin evaluation metrics and current status

3.4 Analysis

- **Objectives met so far:** Objective 1 — registration/login, obligation entry with recurrence, and a dashboard with due-soon flagging — is implemented and functions end to end against a live database.
- **Deviation from plan:** the persistence layer moved from the originally proposed JDBC + SQLite to JDBC + MySQL. This preserves the architecture's separation of concerns (UI / domain / persistence) but adds a running MySQL server as an operational dependency, in tension with the proposal's stated goal of a self-contained, easily deployable Java application.
- **Challenges encountered:** per the team's own week-1 progress log, most early effort went into a learning curve rather than pure feature work — first exposure to core Java/OOP conventions, first exposure to JavaFX's event-driven UI model, and first exposure to JDBC's connect–prepare–execute–close lifecycle, on top of designing a layered application structure from scratch.

- **Resolution:** structuring persistence behind dedicated DAO classes (ChoreDAO, DocumentDAO, UserDAO) instead of embedding SQL in UI code kept database logic isolated and independently testable, and let the team iterate on the JavaFX layer without touching query logic.
- **Data-model gap:** as noted in 2.1.3, the `chores.document_path` column that is meant to connect an obligation to its source document is never populated by the current UI, so the Chores and Documents tabs function as two unconnected features today. Wiring this link is a prerequisite for the OCR feature, not just a nice-to-have.
- **Security notes:** this review found two issues worth flagging before further development. First, `DBConnection` stores the MySQL username and password directly in source code, which is now version-controlled and public on GitHub; this should be externalized (an environment variable or a local, `.gitignore`-excluded config file) and the exposed credential rotated. Second, `UserDAO` stores and compares passwords as plain text in the `users.password` column with no hashing; a proper password hash (e.g., `bcrypt` or `PBKDF2`) should be used before this application handles any real personal documents.

3.5 Risk Assessment

A few risks are worth tracking explicitly heading into the OCR milestone:

- **OCR accuracy risk:** Tesseract-based OCR on scanned or photographed documents is sensitive to skew, lighting, and font; accuracy on the team's own test documents may not generalize to the varied scans a pilot user submits, which is exactly why the design always requires user confirmation before creating a chore from an OCR read.
- **Deployment risk:** requiring a running MySQL server (rather than the originally proposed embedded SQLite file) makes the application harder to hand to a pilot user who does not already have MySQL installed and configured.
- **Regression risk:** with no automated test suite or CI yet, changes made while building the OCR pipeline could silently break the existing chore/recurrence logic; this risk grows as more modules are added.
- **Team bus-factor risk:** with a three-person team and a single shared codebase under one package, uneven familiarity with any one module (e.g., the JavaFX layer versus the DAO layer) could slow progress if a member is unavailable near a deadline.

Chapter 4: Conclusion

4.1 Summary of Work

LifeAdmin set out to close the document-action gap left by locker services such as DigiLocker — storing a document without tracking the deadline tied to it — by building a JavaFX/JDBC desktop application that tracks personal administrative obligations and, eventually, reads renewal and EMI dates directly off uploaded documents. Work to date has delivered the core obligation-tracking workflow — authentication, obligation CRUD with recurrence, and a dashboard that flags items due soon — which together satisfy Objective 1. The document module currently supports manual upload, storage, and retrieval; the OCR-based automatic date-detection feature that is the project's main differentiator, along with the pilot validation planned for Objectives 2 and 3, remain the primary work ahead.

4.2 Key Findings

1. **Finding 1:** structuring persistence behind DAO classes early made it straightforward to add recurrence handling (completeChore's automatic next-cycle insert) without touching UI code.
2. **Finding 2:** a large share of first-iteration effort for a team new to Java, JavaFX, and JDBC goes into the platform's learning curve rather than the product's core logic — worth planning for in later milestones.
3. **Finding 3:** pivoting the persistence layer from SQLite to MySQL mid-project was manageable because the DAO layer isolated SQL from the rest of the app, but it reintroduces a server dependency that the original proposal specifically tried to avoid for ease of deployment.
4. **Finding 4:** the schema already anticipates a chore-to-document link (chores.document_path) that the UI has not yet wired up, meaning the data model is ahead of the UI on the one connection the OCR feature will depend on most.

4.3 Future Work and Recommendations

1. Implement the document-scanning/OCR pipeline (Tess4J) against pre-defined regions for at least one real document template (e.g., an insurance-renewal notice or an EMI schedule), with mandatory user confirmation before a chore is created — this is the project's core remaining scope.
2. Wire the existing chores.document_path column (or a proper foreign key) to the documents table so an OCR-created chore is traceably linked back to its source document.
3. Build the background reminder scheduler (e.g., a ScheduledExecutorService) with configurable lead times (30/7/1 days), rather than relying only on the in-dashboard "due soon" flag.
4. Set up CI (GitHub Actions) with JUnit tests covering the DAO layer, recurrence logic, and — once built — OCR-region extraction.
5. Externalize database credentials out of source code, rotate the currently committed MySQL password, and hash user passwords (e.g., bcrypt) instead of storing them as plain text.
6. Revisit the SQLite-versus-MySQL decision against the proposal's deployability goal, or explicitly document the trade-off if MySQL is being kept.
7. Run the planned pilot with 10–20 users to begin collecting on-time completion and OCR-acceptance data.

4.4 Final Remarks

Most of the engineering challenge in LifeAdmin so far has been foundational — learning Java, JavaFX, and JDBC as a team and getting a clean, layered obligation-tracking workflow working end to end — rather than the document-scanning feature that originally motivated the project. With that foundation in place, the clear priority for the next milestone is the OCR-based date-detection pipeline that turns an uploaded document into a confirmed, tracked deadline: the capability that actually differentiates LifeAdmin from a plain to-do list or a plain document locker.

4.5 Acknowledgements

The team thanks Dr. Raghav B. Venkataramaiyer for guidance on the project's scope and evaluation criteria under UCS503 — Software Engineering, and acknowledges the open-source Java, JavaFX, JDBC, and (planned) Tesseract/Tess4J projects that LifeAdmin is built on.

Appendix A: Source File Summary

For reference, the table below summarizes the responsibility of each Java source file in the repository at this milestone:

File	Responsibility
ChoreApp.java	JavaFX entry point; login/register screen, tabbed dashboard, add-chore form, Documents tab, and all event handlers.
Chore.java	Plain data model for an obligation: id, title, due date, category, done flag, owning user, recurrence.
ChoreDAO.java	CRUD and query logic for chores: addChore, getAllChores, getChoresDueSoon, markDone, deleteChore, completeChore (with automatic recurrence roll-forward).
User.java	Plain data model for a logged-in user (id, username).
UserDAO.java	createUserTable, register, and login logic against the users table.
Document.java	Plain data model for an uploaded document (id, title, file path, upload date).
DocumentDAO.java	createTable, addDocument, getAllDocuments, deleteDocument logic against the documents table.
Database.java	Creates the chores table schema on startup.
DBConnection.java	Central JDBC connection factory for MySQL; currently holds the DB credentials as hardcoded constants (see 3.4).

Table A.1: Java source files and their responsibilities

Bibliography

[1] K. Garg, Sanvi, and L. Singh. LifeAdmin: A Personal Administrative Management Platform. Project Proposal for UCS503P, Thapar Institute of Engineering and Technology, Sept. 2026.

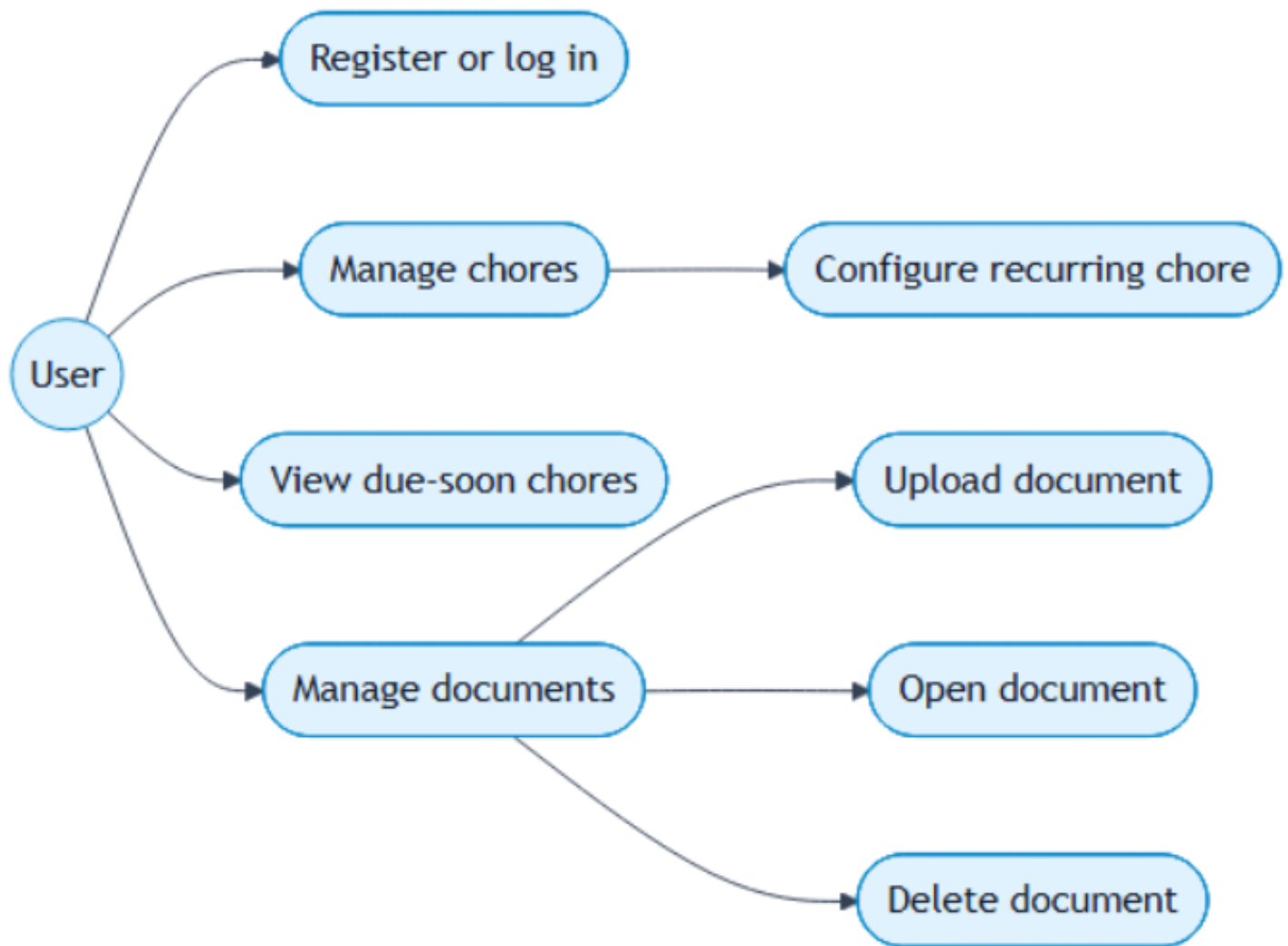
[2] K. Garg, Sanvi, and L. Singh. LifeAdmin: Progress Report (Week 1). Thapar Institute of Engineering and Technology, 2026.

[3] Oracle. JavaFX Documentation. Accessed 2026.

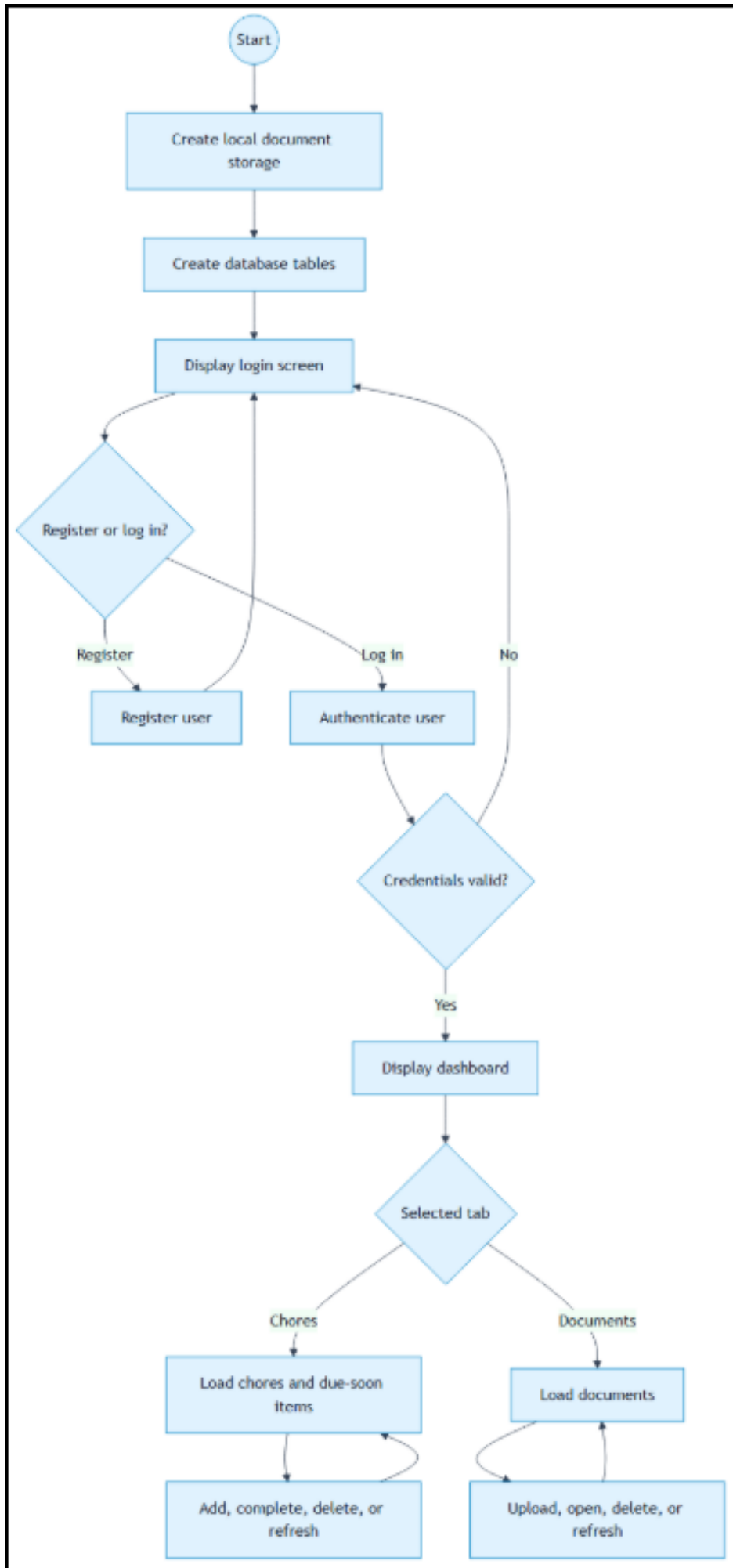
[4] Oracle. Java Database Connectivity (JDBC) API Documentation. Accessed 2026.

- [5] Tesseract OCR / Tess4J. Open-source OCR engine documentation. Accessed 2026.
- [6] MySQL 8.0 Reference Manual. Oracle Corporation. Accessed 2026.

Use Case Diagram



Activity Diagram



Class Diagram

